

packages/graph-core/src/__tests__/parsers.test.ts

Kind: added · Baseline: a9c34bfb03fe · Target: NovaDiff · Generated: 2026-05-20T05:58:29.389Z

Overview

A new test file `packages/graph-core/src/__tests__/parsers.test.ts` (lines R1-629) has been added. It imports all 12 parser plugins—Markdown, YAML, JSON, TOML, Env, Dockerfile, SQL, GraphQL, Protobuf, Terraform, Makefile, and Shell—plus `PluginRegistry` and `registerAllParsers`. The file contains a `describe` block for each parser, exercising `analyzeFile`, `extractReferences`, and the `languages` property, and concludes with a registry test that ensures all parsers are registered.

Key changes

- **Imports added** (R1-R14):

```
``ts import { MarkdownParser } from "../plugins/parsers/markdown-parser.js"; import { YAMLConfigParser } from "../plugins/parsers/yaml-parser.js"; import { JSONConfigParser, stripJsoncSyntax } from "../plugins/parsers/json-parser.js"; import { TOMLParser } from "../plugins/parsers/toml-parser.js"; import { EnvParser } from "../plugins/parsers/env-parser.js"; import { DockerfileParser } from "../plugins/parsers/dockerfile-parser.js"; import { SQLParser } from "../plugins/parsers/sql-parser.js"; import { GraphQLParser } from "../plugins/parsers/graphql-parser.js"; import { ProtobufParser } from "../plugins/parsers/protobuf-parser.js"; import { TerraformParser } from "../plugins/parsers/terraform-parser.js"; import { MakefileParser } from "../plugins/parsers/makefile-parser.js"; import { ShellParser } from "../plugins/parsers/shell-parser.js"; import { PluginRegistry } from "../plugins/registry.js"; import { registerAllParsers } from "../plugins/parsers/index.js";
```

``- **Parser tests**: Each `describe` block (e.g., `MarkdownParser`, `YAMLConfigParser`, ...) contains multiple `it` cases covering normal parsing, edge cases, and reference extraction.

- **Registry test** (lines 611-629): verifies that `registerAllParsers` registers 12 plugins and that the registry's supported languages include all expected identifiers.

Impact

- **Regression safety**: The tests provide high-coverage checks for parser logic; any future change that alters expected output will surface immediately.
- **Maintainability**: Centralizing parser tests simplifies adding new parsers or updating expectations.
- **CI cost**: Running 12 parser suites adds modest runtime; no external I/O is involved.

Risks & follow-ups

- **Test determinism**: Some assertions rely on exact string output (e.g., `stripJsoncSyntax`). Ensure consistent environment settings to avoid flakiness.

- **Expectation drift:** If parser implementations evolve, expected arrays or strings may need updating; review failures for intentional changes.
- **CI load:** Monitor overall test duration; consider parallel execution or test splitting if it becomes a bottleneck.